

From Zero to Sixty at the Speed of RAG: Improving YAML Recipe Generation via Retrieval

Farima Farmahinifarahani

J.P. Morgan AI Research

Palo Alto, USA

farima.farmahinifarahani@jpmchase.com

Salwa Alami

J.P. Morgan AI Research

London, UK

salwa.alami@jpmchase.com

Petr Babkin

J.P. Morgan AI Research

Palo Alto, USA

petr.babkin@jpmorgan.com

Xiaomo Liu

J.P. Morgan AI Research

New York, USA

xiaomo.liu@jpmchase.com

Abstract—LLMs have been shown to match or even exceed the performance of specialized Deep Learning models on code generation tasks for general purpose imperative languages, such as Python, Java, C++, and Rust. Conversely, there is only limited work investigating whether such impressive out of the box generalization transfers onto less ubiquitous domain-specific languages, which are often declarative, based on XML, JSON, or YAML. To bridge this gap, we explore the capabilities of LLMs for composing code automation recipes without resorting to any form of task-specific finetuning. We experiment with two GPT versions and CodeLlama-13b-Instruct, and in our experiments, we find that after extensive prompt engineering and chain-of-thought prompting, these models' accuracy in recipe selection does not go beyond $\approx 30\%$. For parameter filling of YAML recipes, the accuracy of these models remains below $\approx 50\%$. However, by decomposing the task into two stages: dense retrieval and generative slot filling, and while still keeping our setup training-free, the models are able to attain an accuracy in a range of $\approx 50\%$ to $\approx 67\%$ in recipe selection, and $\approx 60\%$ to $\approx 76\%$ in parameter filling. Our study sheds light on the capabilities of LLMs in generating scripts for less widespread languages and opens up avenues for future research.

Index Terms—code generation, code refactoring, LLM

I. INTRODUCTION

The introduction of large language models proved disruptive for many fields, including software engineering. Numerous studies demonstrated out of the box performance of pretrained code LLMs, such as Codex [1], to be competitive with state of the art on a variety of coding tasks, ranging from code search and clone detection to program synthesis and automatic program repair [2]. As a result of this impressive performance, LLM-powered assistive coding tools, such as Copilot, Code Whisperer, and Code Compose became ubiquitous in the enterprise setting, as a means of improving developer productivity [3]. With a new code LLM introduced seemingly every month, the spotlight remains squarely on supporting code authoring in the popular general purpose imperative languages, such as Python, Java, and C++. Incidentally, the capabilities of LLMs at declarative domain specific code such as XML, JSON, and YAML, which are relied upon by various

configuration and software lifecycle management tools (such as maven and jules) remains relatively understudied, despite these declarative languages making up considerable portions of industrial codebases. To our knowledge, one of the few works is that by IBM Research and Red Hat [4], which evaluates the ability of LLMs in generating Ansible YAML IT automation playbooks. In a similar spirit, our work seeks to understand the ability of LLMs to generate YAML recipes for the code refactoring automation framework OpenRewrite [5].

OpenRewrite is an open source framework for large-scale code refactoring automation to help with API migrations, vulnerability patches, and maintaining stylistic consistency. Internally, it works by modifying the formatting-preserving semantic tree representation of the code, which is then converted back to code. Core code transformations are implemented by the OpenRewrite API and to achieve a specific refactoring task, they are commonly parametrized and bundled together in a special declarative YAML file called a *recipe*. The official OpenRewrite catalog includes over a thousand individual recipes, targeting code written in imperative languages, like Java, Python, and Kotlin, as well as declarative languages like Jenkins, Kubernetes, Gradle, and Terraform.

Recipes vary in complexity: from basic ones performing a single transformation without parameters (like converting imports to static) to more complex ones consisting of multiple recipes with parameters – common in framework migrations. The diversity of OpenRewrite recipes make it a challenging domain for research on the capabilities of LLMs at generating domain-specific code in a declarative language. In addition, this line of work is relevant to the industry, as automating recipe creation can help reduce the amount of time and alleviate the need for specialized expertise currently required to compose novel recipes by hand.

Our key contributions amount to reporting on two sets of experiments using three state-of-the-art LLMs: CodeLlama-13b-Instruct, GPT4o-mini, and GPT4o. First, using various prompting techniques, we ask these models to generate a complete YAML recipe, which was shown to be lacking in

```

type: specs.openrewrite.org/v1beta/recipe
name: com.yourorg.DeleteMethodArgumentExample
displayName: Delete method argument example
recipeList:
  - org.openrewrite.java.DeleteMethodArgument:
      methodPattern: com.yourorg.Visuals foo(int,int)
      argumentIndex: 0

```

Fig. 1: Example of an OpenRewrite recipe usage YAML [5]

terms of performance, and its generations were observed to be likely to contain hallucinated content. Second, we engineer a retrieval augmented generation (RAG) [6] pipeline for greater control of the generation and show how it helps alleviate the above issues by keeping the LLMs grounded to an existing recipe template. Lastly, we discuss remaining limitations and offer potential directions for future work.

The rest of this paper is organized as follows. In Section II, we show the schema of a YAML recipe and formulate the generation task, provide a motivating example, and review the related work. Subsequent sections detail our experimental setup (Section III) and discuss the findings of the two sets of experiments (Section IV). We close with a discussion of limitations and threats to validity, followed by the conclusions and future work.

II. PRELIMINARIES

A. Background and Problem Formulation

OpenRewrite [5] API implements a set of core recipes that can be included declaratively in a YAML file. If a recipe accepts parameters, those and their values are defined in the YAML script. Another usage of YAML scripts is combining multiple recipes to form a composite recipe. An example of an OpenRewrite YAML is shown in Figure 1. It contains the following fields:

- **type**: framework’s specific constant.
- **name**: recipe’s unique identifier.
- **displayName**: human-readable name for the recipe.
- **recipeList**: used to include previously implemented recipes that achieve specific code transformations. Since the YAML file is also called a recipe, for ease of reference, we refer to recipes appearing under *recipeList* as *subrecipes*.

Each subrecipe included under the *recipeList* is itself a YAML object with its own fields for arguments. The YAML script can contain additional fields, such as description and tags.¹

Given the above YAML structure, we are centrally interested in generating the part under *recipeList* which implements the recipe’s actual functionality, more so than other top level fields, some of which never change or can be arbitrarily populated from the prompt. Therefore, our problem can be formulated as follows: given a user query Q , consisting of natural language instructions defining the code transformation (refactoring) task

¹The complete list is defined on the following page: <https://docs.openrewrite.org/reference/yaml-format-reference>

```

type: specs.openrewrite.org/v2
name: RemoveFirstArgumentFromFooMethod
description: Remove the first argument of method foo
recipeList:
  - org.openrewrite.java.ChangeMethodDeclaration:
      match:
        type: java
        name: foo
        qualifiers: []
        parameters:
          - type: int
          - type: int
      replace:
        parameters:
          - type: int

```

Fig. 2: A recipe generated by GPT-3.5-Turbo and GPT-4o-mini in response to query: “remove the first argument of method foo. This method accepts two int parameters. Only change this for methods in com.yourorg.Visuals namespace”.

t , our target is a function $RecipeGen(Q^t) \rightarrow R^t$, which given Q , generates a YAML recipe R that when executed, performs t . We parametrize the generation function with a transformer decoder LLM that generates R conditioned on Q .

B. Motivating Example

Given the above problem formulation, it is straight-forward to test how well state of the art LLMs can perform this task out of the box. In our preliminary experiments with GPT-3.5-Turbo we quickly found that even though generated recipes look plausible, most of the time they reference non-existent subrecipes, incorrectly employ the arguments, and even violate the YAML format, which renders them unusable in a software engineering setting. The example in Figure 2 demonstrates some of these failures: the recipe `java.ChangeMethodDeclaration` is not defined in OpenRewrite and the LLM populated it with arbitrary parameters. Additionally, the value for the type field does not correspond to a real API specification. In this work we show that these errors can be fixed by conditioning the generation on a correct recipe template. In Section III we formally evaluate GPT-4o and GPT-4o-mini in both end-to-end and retrieval augmented generation settings.

C. Related Work

Since here we target generating refactoring scripts using LLMs, one related line of research is using LLMs for code refactoring. Poldrack et al. [7] show that GPT-4 assisted code refactorings can improve several Python established code quality metrics such as maintainability index, mean cyclomatic complexity and Halstead bugs. In another study, researchers at Microsoft Research [8] propose to fix code quality issues with LLMs (GPT-3.5-Turbo). In their work, prompt is augmented with a static analysis tool report, the flagged source code, and instructions to fix the issues from online documentations. Their pipeline consists of two LLMs: proposer who proposes candidate revisions, and ranker whose job is to rank proposer’s suggestions.

Another related line of research are LLMs for Code, and particularly, YAML snippets generation. Several works have been done in context of general-purpose programming languages code generation with various CodeLLMs developed; some notable examples are Codex [1], Code LLaMA [9] and CodeGen [10]. More specifically targeted at YAML scripts generation, authors from Redhat [4] propose fine tuned models based on CodeGen that are specific to Ansible-YAML generation. They use different prompt types for natural language to YAML translation and show that their fine-tuned models perform better than or similar to the state-of-the-art code generation models.

While similar to the above work, our problem is a novel one as it falls in between the two mentioned research areas: we explore the capabilities of LLMs to generate OpenRewrite YAML scripts that perform code refactorings. We draw inspirations from these works, as well as from proven methods of prompt engineering such as chain-of-thought prompting [11] and retrieval augmented generation (RAG) [6], [12], [13].

III. EXPERIMENTS SETUP

A. Dataset

Our experiment setup requires a dataset consisting of user queries explaining the code refactoring task, and the corresponding refactoring yaml recipe. Ideally, we would like this dataset to consist of (query, yaml recipe) pairs that were not included in the training data of either of the studied LLMs.

Our starting dataset for evaluating recipe generation consisted of just under 20 seed examples of recipes and their natural language descriptions, provided by the business stakeholders. However, to expand on our dataset and help increase the statistical validity as well as the reproducibility of our findings, we leveraged the public catalog of OpenRewrite. Our goal was to curate a dataset of YAML recipes defined to invoke recipes that accept arguments (parameters) accompanied with user queries that explain the task that the YAML script accomplishes. The OpenRewrite catalog consists of over a thousand recipes, nearly half of which are defined programmatically and exposed as part of the framework’s core API. The remaining recipes are defined in YAML by importing either API-implemented or other YAML-defined recipes as *subrecipes*. YAML-defined recipes can form complex hierarchies, potentially comprising multiple nested subrecipes. It should be noted that only API-defined recipes can be parametrized with arguments, whereas complex YAML recipes largely serve as containers aggregating individual recipes for some fixed high-level task, such as migration to Spring Boot 3. In this work we focus on the task of generating YAMLS that import a single programmatically defined recipe, instantiated with arguments as specified in the user’s natural language query. We do not consider recipes without arguments or recipes consisting of multiple subrecipes. As a result of this selection, we obtained an initial set of 217 recipes for annotation.

One direction we explored was synthetic generation of user queries by paraphrasing recipe descriptions found in the web documentation using an LLM. However, this approach could

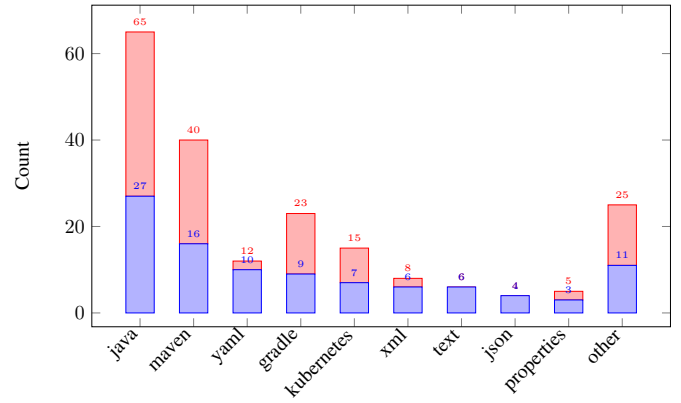


Fig. 3: A breakdown by language of recipes annotated (in blue) vs total (blue + red).

overestimate the actual performance due to the possibility of information about the target recipe leaking into the query, and unrealistic phrasing of the queries due to the use of the LLM.

We thus curated the dataset manually, with the help of 5 NLP researchers and 3 infrastructure engineers who wrote user queries that perform a refactoring task that can be attained using a recipe. Since authoring the queries requires adequate understanding of the recipe’s underlying technology, we allocated recipes covering common languages such as java, yaml, xml, and json among NLP researchers, and more specialized build system and infrastructure recipes such as maven, gradle, and kubernetes – to the engineers. Among each set, annotators were free to choose to develop a (query, recipe) pair based on a recipe that they are most informed about. This annotation process also included filling in the parameter values for recipe parameters, and/or omitting optional parameters not mentioned in the query. Two of the authors then did two independent passes over the annotations, ensuring the consistency of queries with the recipes, and resolving discrepancies through an open discussion. This process resulted in 100 gold (query, yaml recipe) pairs. The statistics of these annotations by language are given in Figure 3.

B. Experiments Design

Our main goal is to investigate the capabilities of various LLMs in generating YAML refactoring recipes. For this, we explore various prompt templates and generation techniques. In one set of experiments, which we call *baseline* experiments, we explore standard prompting techniques. We then perform another set of experiments where we explore *Retrieval Augmented Generation (RAG)* under two different scenarios. We explain each experimental setup in this section.

Baseline Experiments. Here, we adhere to prompting LLMs using two types of prompts:

1) *Zero-shot prompt:* the LLM is asked to *generate the OpenRewrite YAML recipe script that performs a given code refactoring task*. The goal is to investigate the performance of the studied LLMs in generating a complete and correct yaml script given a task description (user query), without giving any

further instructions. This is the most straightforward way of approaching this problem that is worth exploring.

2) *Chain-of-Thought (CoT) prompt* [11]: the LLM is given step by step instructions for generating the target YAML script: *Provided with a generic YAML recipe template, LLM is asked to first, pick the correct yaml (sub)recipe that can be used to accomplish the given task, and then, assign the relevant parameter values to the parameters of the recipe picked in step 1 so that the recipe performs the given task.*

This prompt format is inspired by the developers thought process for developing such YAML scripts: they typically first search among the available recipes to find the one that can be used for their target task, and then, they fill in the relevant values for the parameters that this recipe accepts. With this prompt, we investigate if providing a generic YAML template for recipe generation accompanied a step-by-step generation instruction, would help in generating more accurate scripts compared to the zero-shot setting.

RAG Experiments. With the baseline experiments, we are expecting LLMs to find the most applicable recipe (with respect to the user query) and fill in its parameter values accordingly. However, given that the official recipes form a relatively small closed set and we have access to a description about their usage (via their documentation [5]), here we have the opportunity to further assist the LLMs in the recipe selection step: the task of choosing the relevant recipe could be offloaded to a retrieval system, while the LLMs’s capabilities could be leveraged to properly structure and populate that recipe’s parameters to form the final YAML script.

To test this idea, we perform a set of *Retrieval Augmented Generation (RAG)* experiments. The generation pipeline in RAG experiments consists of two phases: *Retrieval* and *Generation*. During the *Retrieval* phase, we embed the user query using a sentence transformer model designed for encoding sentences and short paragraphs [14]. This embedding is then used to calculate the similarity between the user query and the available OpenRewrite recipes (1182 total) which are stored as pairs of $\langle \text{RecipeDescriptionEmbedding}, \text{RecipeYAMLSnippet} \rangle$. The top-k matching pairs are retrieved and their corresponding YAML snippets are passed on to the next phase.

More formally, Given a pool of recipes and their descriptions $\{(R_1, D_1), (R_2, D_2), \dots, (R_n, D_n)\}$ where each tuple represents a recipe and its description, for a user query Q , we retrieve a set of *Top-K* recipes in terms of dot-product similarity $\text{DotSim}(D_k, Q)$, where $k \in \{1..K\}$ and Rec_k is classified as k_{th} most similar recipe to R .

During *Generation*, the final prompt containing the user query (summarizing the task description) is augmented with the Top-K fetched recipe(s), and a description of their *parameters*. The configuration of K depends on the prompt type which we describe in the rest of this section.

The parameters description helps LLMs in associating each parameter with its appropriate value from the user query. Since most of the OpenRewrite recipes’ parameter names follow a camel case format, we derive parameters descriptions by

You are provided with an OpenRewrite YAML script, list of the parameters this YAML script accepts and their definitions, and a code refactoring task description. Your task is to fill in the relevant parameter values for each parameter in the given script so that the resulting YAML script performs the given task.

The task description is: "Refactoring task’s user query".

The YAML script is:

Retrieved YAML template

Here is the list of parameters in this script and their definitions: Reference to each parameter the recipe accepts and their definitions.

You should put the value for each parameter after the : sign in front of the parameter name. Parameters only appear in the recipeList section of the script. Only provide values for parameters whose values are mentioned in the task description. Remove any parameter whose value is not mentioned in the task.

Your response should only be the final YAML script, with no other text. If you do not have information about this task, output: "Cannot generate recipe."

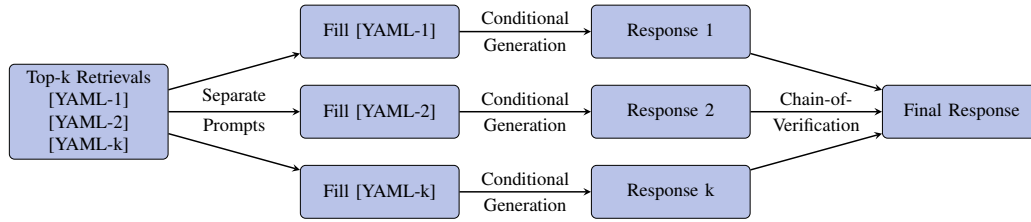
Fig. 4: RAG-k-Separate Prompt Template

doing a camel case split over parameter names. This helps in reducing the prompts lengths (as recipes can accept several parameters and each parameter description can be long) and can also expand the usability of the approach where parameters descriptions are not available.

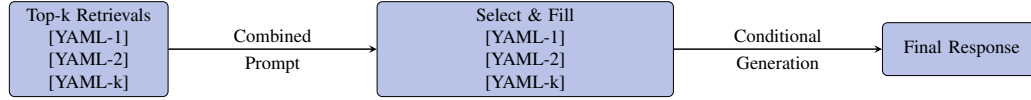
Two prompt types are explored in RAG experiments:

1) *RAG-k-Separate prompts*. Here, the prompt is only augmented with one YAML recipe template from among the Top-K retrievals. An obvious choice here is the Top-1 retrieved recipe; however, there can be cases where the most relevant recipe does not rank at Top-1. In fact, using our chosen embedding model, we observed the following retrieval accuracies: Top-1: 53%, Top-2: 62%, and Top-3: 70%, showing that there can be cases where the relevant recipe does not rank at Top-1. Therefore, we construct K prompts using each of the Top-K retrieved recipes and run each through LLMs.

The output of this step is K fully generated YAML recipes. To decide which of these scripts is the one matching the user’s need, we again engage the LLM: for each of the parameters of a script, we ask the LLM to verify whether the parameter or its value is *implied or mentioned* in the user query. We then choose the script that has the highest score in terms of the fraction of its parameters being mentioned in the query. The intuition here is that if there are multiple recipes whose descriptions are similar to the user query, we can then rely on their parameters and/or parameter values as means of assessing recipe relevance because these values are also often, explicitly or implicitly, mentioned in the query. In case of ties, the subrecipe retrieval score is used to rank the scripts. Figure 4 shows the prompt template.



(a) RAG-k-Separate prompts. Each of the top-k retrieval results are used independently for conditional generation. Final response is chosen via scoring them.



(b) RAG-k-Combined Prompt. All of the top-k retrieval results are used together for conditional generation. The LLM has to simultaneously choose among recipe templates and fill in parameters.

Fig. 5: Schematic illustrations of the two prompt pipelines used in RAG experiments

This whole process, illustrated on the top part of Figure 5, is inspired by Chain-of-Verification (CoV) method [15] where LLMs are instructed to come up with verification questions about their responses, and use answers to those questions to improve their initial answers. Here, we ask LLMs to verify each generated YAML based on its relevance to parameters mentioned in the user query, and use the results to select the most suitable generation.

2) *RAG-k-Combined Prompt*. In this setting, instead of separating the script generation and ranking to two prompts, we construct one prompt and carry out both steps at one pass.

Here, the prompt is augmented with all Top-K retrieved YAML recipe templates and their parameters description. Given this information, LLM is asked to first choose the recipe template that is most relevant to the user query, and then, to fill in the parameter values for the chosen recipe. This process is depicted on the bottom part of Figure 5, and the prompt template is shown in Figure 6.

IV. EVALUATION RESULTS

Given that YAML recipes are complex hierarchical structures, in evaluating them, it is important to take this structure into account. For example, measuring the exact match or the textual overlap with the ground truth recipe is suboptimal when there are multiple valid ordering permutations of elements such as subrecipes or arguments, or when incorrectly structured recipe may still result in a high textual overlap. Secondly, a coarse evaluation can be limiting our understanding of the strengths and weaknesses of the models being evaluated. Thus, we measure the correctness of the predicted subrecipe name and its arguments separately.

For all RAG experiments, we set $K=3$ and report results for Top-3 retrieved recipes.

A. Subrecipe Selection Accuracy

The very first and a key part of a YAML recipe script is the subrecipe invoked by it: it determines the main functionality of the recipe, and also drives the parameter filling process.

You are provided with 3 OpenRewrite YAML scripts, list of the parameters each YAML script accepts and their definitions, and a code refactoring task description. Your task is to select the YAML script most relevant to the task description, and to fill in its parameter values so that the resulting YAML script performs the given task.
The task description is: "Refactoring task's User query".
Here are the YAML scripts, their parameters, and descriptions of parameters. Each recipe is separated by ###.
###
First YAML template and its parameters description
###
Second YAML template and its parameters description
###
Third YAML template and its parameters description
First, select the most relevant YAML script to the task description, and then, fill in the parameter values for this script according to the task description. You should put the value for each parameter after the : sign in front of the parameter name. Parameters only appear in the recipeList section of each script. Only provide values for parameters whose values are mentioned in the task description. Remove any parameter whose value is not mentioned in the task.
Your response should only be the final YAML script, with no other text. If you do not have information about this task, output: "Cannot generate recipe."

Fig. 6: RAG-k-Combined Prompt Template

Therefore, it is worthwhile to first evaluate the subrecipe selection accuracy for each experimental setting. Accuracy here measures in how many cases, out of the total number of cases in dataset (100 in our dataset), the subrecipe used by final generated YAML script matches the subrecipe mentioned in the groundtruth YAML script. In other words:

TABLE I: Subrecipe selection accuracy across the experiments

	CodeLlama-13b-Instruct	GPT-4o-mini	GPT-4o
Baseline-Zero Shot	N/A	0.03	0.31
Baseline-CoT	0.02	0.04	0.25
RAG-k-Separate	0.57	0.58	0.49
RAG-k-Combined	0.52	0.63	0.67

$$\text{SubrecipeSelectionAccuracy} = \frac{\text{NumCorrectSubrecipes}}{\text{Num}_{\text{Total}}}$$

Table I² presents subrecipe selection accuracy results.

Starting with the fully generative baselines, we see subrecipe selection only achieves 31% by the strongest model and is basically non-existent for the smaller models, which confirms our earlier hypothesis. Somewhat disappointingly, and counter to our intuition, chain-of-thought prompting does not significantly improve the groundedness of generated subrecipe names. When it comes to retrieval-augmented configurations, the first observation from this table is the significant improvement the RAG solutions bring compared to non-RAG ones across all models. With CodeLlama-13b-Instruct and GPT-4o-mini we see an improvement from 0% accuracy to $\approx 60\%$, and with GPT-4o, the RAG solutions offer an almost two-fold improvement compared to non-RAG ones. We also see that thanks to RAG-k-Combined Prompt, GPT-4o has achieved a subrecipe selection accuracy that nearly matches the 70% Top-3 recipe retrieval accuracy discussed in Section III-B. This suggests the strong LLM not only benefits from retrieval conditioning but is also robust to its errors.

Looking more closely at each model’s performance in RAG solutions, we see that the best subrecipe selection result is achieved by GPT-4o in RAG-k-Combined Prompt setting. It is followed by GPT-4o-mini in the same setting, and then CodeLlama-13b-Instruct in RAG-k-Separate prompts setting. This can imply that when choosing the most relevant subrecipe, GPT models performed better in an end-to-end RAG setting where the selection of the relevant recipe (from among a limited pool of three) as well as the generation of the full YAML is done using one single prompt. On the other hand, CodeLlama-13b-Instruct performed better in a more siloed setup: where it was first prompted to fill in the parameters for each retrieved recipe independently, and then, as a separate step, asked to verify each generation against the user query, in the style of Chain-of-Verification.

B. Parameter Filling Accuracy

To complete our evaluation of end-to-end YAML recipe generation, we also evaluate the generation capabilities of the studied LLMs by assessing their parameter filling accuracy.

²N/A in this table means that none of the generations followed a valid YAML recipe format, and therefore, we could not assess accuracy here. The same applies to Table II.

For each subrecipe, we flatten their argument values (if nested) for both the generated and ground truth YAMLs. This enables us to look at the precision and recall side of the prediction and it allows for partial credit even in the case where the subrecipe itself was chosen incorrectly. Furthermore, when comparing predicted and ground truth argument values we account for the following benign scenarios:

a) *Values predicted for default/optional arguments are ignored:* Many recipes contain arguments that are default or optional and are not mentioned in the user’s query. We identify such arguments if their value in the reference recipe are either booleans or None.

b) *Partial and near-match:* A significant number of parameters involve strings such as file paths or glob expressions that allow for some variation without impacting the semantics. We compute a Levenshtein ratio using `fuzzywuzzy`³ between the predicted and ground truth value and threshold them by 0.8 for a near-match and by 0.3 for a partial match.

c) *Nested parameter values:* YAML allows values to be composite objects rather than primitive types. When we detect a discrepancy when one of the compared values is hierarchical while the other is not, we attempt to flatten the hierarchical one into a string and then compute the Levenshtein ratio.

d) *Plausible parameter value:* Many recipes achieve similar functionality via slightly differently named arguments that serve the semantically equivalent purpose. In cases like these we check if the matching value is contained under an argument with a different name and assign partial credit. An example exhibiting the above issues is shown in Figure 7.

We aggregate these metrics to report both *parameter*-level and *recipe*-level average accuracy, as indicated in Table II. As Table II shows, there’s a noticeable increase in the generation accuracy when comparing RAG experiments (shown in the bottom two rows) with the baseline experiments (shown in the top two rows). In all RAG settings, recipe-level accuracy is above 0.6 and parameter-level accuracy is above 0.55. The best recipe-level accuracy is achieved by GPT-4o in RAG-k-Combined Prompt setting (0.7573), followed by GPT-4o-mini in the same setting (0.7510). In parameter-level accuracy, again, the highest performance is achieved in RAG-k-Combined Prompt setting, with GPT-4o-mini’s accuracy of 0.6885, followed by GPT-4o’s accuracy of 0.6749.

An interesting observation is that while GPT-4o and GPT-4o-mini are performing better in RAG-k-Combined Prompt setting, and CodeLlama-13b-Instruct is performing better in RAG-k-Separate prompts. This is a similar observation to what we saw in subrecipe selection: GPT models performing better in an end-to-end one prompt setting, while CodeLlama-13b-Instruct performing better in the Separate setting.

Overall, all models show an improvement in range of $\approx 15\%$ to $\approx 30\%$ in RAG settings compared to their best performance in each category of baseline settings. This is an indication of how grounding LLMs to a relevant template can improve their generation capabilities for YAML recipes.

³<https://pypi.org/project/fuzzywuzzy/>

TABLE II: Parameter filling accuracy aggregated at the level of recipes as well as individual parameters

	CodeLlama-13b-Instruct		GPT-4o-mini		GPT-4o	
	Recipe	Param	Recipe	Param	Recipe	Param
Baseline-Zero Shot	N/A	N/A	0.2012	0.2281	0.3854	0.3862
Baseline-CoT	0.4804	0.4523	0.4623	0.4554	0.4806	0.4745
RAG-k-Separate	0.6807	0.6089	0.6983	0.6366	0.7075	0.6216
RAG-k-Combined	0.6053	0.5559	0.7510	0.6885	0.7573	0.6749

<pre> - org.openrewrite.text.FindAndReplace: find: blacklist replace: banned regex: null caseSensitive: null multiline: null dotAll: null filePattern: '**/*.java' </pre>	<pre> - org.openrewrite.concourse.ChangeValue: oldValue: blacklist newValue: banned fileMatcher: - fileType: java </pre>
---	---

Fig. 7: An example of partially matching argument values among distinct but semantically similar recipes, matching the query “replace all occurrences of the string blacklist in all java files with banned”. Values highlighted in red are discarded, whereas ones in green and yellow are assigned partial scores of 0.5 and 0.25 respectively.

C. Error Analysis

The 100 evaluated recipes contain an average of 2 and a maximum of 6 parameters for a total of 197 across all recipes. Figure 8 shows the breakdown of different prediction statuses and error types for four configurations: the strongest non-RAG approach – GPT-4o CoT, versus the best RAG configuration for each of the three LLMs.

As the non-RAG configurations have the lowest subrecipe selection accuracy, across all 3 models, their parameter predictions also have the lowest number of exact matches (50 for the strongest among them, GPT-4o CoT). This amounts to mere 15 overall exact subrecipe section match. The inclusion of partial matches increases the number to 58. Further 72 predictions of parameter values are categorized as plausible as they match the values found in the gold recipe under a different parameter name (due to the incorrect recipe retrieval). This implies strong slot filling ability, notwithstanding the lack of grounding in the generation of recipe names, pointed out earlier.

On the other hand, the strongest among the RAG configurations, GPT-4o RAG-k-Combined Prompt shows the highest exact and combined exact + partial match of 104 and 116, respectively. This translates into 48 overall exact subrecipe section matches, more than triple the number achieved by the same model in a non-RAG setting. The improved subrecipe selection accuracy also means more than 2x reduction in the number of plausible parameter predictions, which are converted into exact and near matches.

GPT-4o-mini RAG-k-Combined Prompt shows a similar distribution of errors, with a larger fraction of plausible predictions but also a smaller fraction of predictions without

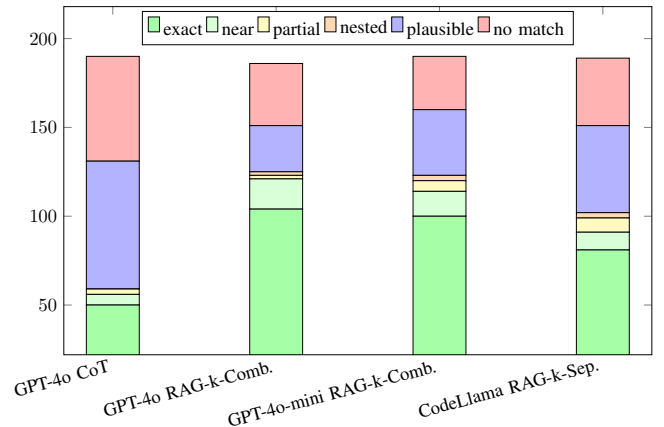


Fig. 8: Prediction & error types breakdown

any match. The number of overall exact subrecipe section matches is only slightly behind GPT-4o, at 46.

Finally, the smallest of the models, CodeLlama-13b achieves 85 exact and 95 combined exact and partial parameter matches, resulting in 41 overall exact subrecipe section matches with the ground truth. Unlike the other two models, it achieves its best performance in the RAG-k-Separate prompts configuration. We attribute this to the model’s limited ability to choose among multiple conditioning contexts present in the RAG-k-Combined Prompt setting.

V. THREATS TO VALIDITY

In the absence of a large scale public corpus of real-world examples⁴ of user queries and their YAML recipes, we manually developed a set of user queries and their corresponding YAML recipes. While the (user query, recipe) pairs were independently reviewed by two authors of the paper to make sure they are correct and close to developer-written ones, there might be some deviation from real-world data. This might cause some divergence in accuracy results observed in our experiments from what happens in real-world scenarios.

In our evaluation, we aimed to approximate real usage by instantiating known recipes for hypothetical code modification scenarios using plausible parameter values, taking extra care to ensure they do not duplicate the examples provided in OpenRewrite documentation. Such an approach does not allow for extrapolation of performance onto some unknown universe of recipes and uncommon usage scenarios.

While the task of generating YAML recipes is motivated by a real technology need for automating code changes, the research focus of this work is on evaluating the capabilities of LLMs at domain-specific declarative code. Thus, our evaluation is limited to measuring the intrinsic accuracy of generated recipes against the ground truth. It could be beneficial to also evaluate the correctness of downstream code changes from applying the recipes and, benchmarking against a pure code generation approach, bypassing the recipes altogether.

We used a fixed value of k equal to 3 in our experiments, although a comprehensive ablation of this hyperparameter could provide additional insights.

VI. CONCLUSIONS AND FUTURE WORK

In this paper we evaluated the capabilities of LLMs, notable for their general coding proficiency, at generating domain-specific declarative YAML of OpenRewrite recipes. We found that while the out-of-the box performance of state of the art models at this task is not on par with the level of general purpose coding, it can be significantly improved using a retrieval augmentation technique, without the need for in-domain finetuning. Not only does RAG improve recipe generation by state of the art LLMs, it also make it feasible for smaller open source models, such as CodeLlama-13b, to be deployed for this task in production. One direction for future research is to expand the evaluation to other declarative languages and frameworks, with the possible inclusion of less frequently used programming languages.

VII. DISCLAIMER

This paper was prepared for informational purposes by the Artificial Intelligence Research group of JPMorgan Chase & Co. and its affiliates ("JP Morgan") and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the

information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [2] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1482–1494.
- [3] V. Murali, C. Maddila, I. Ahmad, M. Bolin, D. Cheng, N. Ghorbani, R. Fernandez, and N. Nagappan, "Codecompose: A large-scale industrial deployment of ai-assisted code authoring," 2023.
- [4] S. Pujar, L. Buratti, X. Guo, N. Dupuis, B. Lewis, S. Suneja, A. Sood, G. Nalawade, M. Jones, A. Morari *et al.*, "Automated code generation for information technology tasks in yaml through large language models," *arXiv preprint arXiv:2305.02783*, 2023.
- [5] "Rewrite docs," <https://github.com/openrewrite/rewrite-docs>, accessed: 2010-02-05.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [7] R. A. Poldrack, T. Lu, and G. Beguš, "Ai-assisted coding: Experiments with gpt-4," *arXiv preprint arXiv:2304.13187*, 2023.
- [8] N. Wadhwa, J. Pradhan, A. Sonwane, S. P. Sahu, N. Natarajan, A. Kanade, S. Parthasarathy, and S. Rajamani, "Frustrated with code quality issues? llms can help!" *arXiv preprint arXiv:2309.12938*, 2023.
- [9] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [10] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.
- [11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [12] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, and W.-t. Yih, "Replug: Retrieval-augmented black-box language models," *arXiv preprint arXiv:2301.12652*, 2023.
- [13] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [14] "Sentence transformers," <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.
- [15] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," *arXiv preprint arXiv:2309.11495*, 2023.
- [16] D. Kocetkov, R. Li, L. B. Allal, J. Li, C. Mou, C. M. Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf, D. Bahdanau, L. von Werra, and H. de Vries, "The stack: 3 tb of permissively licensed source code," 2022.

⁴We conducted a search among YAMLs contained in The Stack [16] dataset and found with, very few exceptions, all recipes to be copied from the OpenRewrite repository.